

第六章讲稿 v3（整合定稿版，对应 chapter06-presentation-v3.pdf，共 53 页 / 1 小时）

结构：五部分合一、单封面；原“本章讲什么”页与路线图重复，已并入路线图页。为控制在 1 小时内，删去三页（__launch_bounds__、Occupancy API、Compute Sanitizer——材料保留在 part4/part5 draft 中，讲稿相应位置有一句话交代）。

时间分配：开场 1-2 页 3 分 | Part I 3-21 页 19.5 分 | Part II 22-29 页 9 分 | Part III 30-38 页 9.5 分 | Part IV 39-46 页 10 分 | Part V 47-50 页 6.5 分 | 收尾 51-53 页 2.5 分 = **60 分**。

超时保险（按序启用）：□ 第 5 页 SFU 并入第 4 页一句话；□ 第 12 页快讲（第 13 页承担 SIMT 主讲）；□ 第 43 页完整版跳过（纯加分页）；□ 第 49 页低精度压到 60 秒。

每节结构：【口播】口播稿／大白话收束句／【问】预备问答；【必说】= 必说的转场或补充。

第 1 页 | 标题页

【口播】大家好，今天我讲第六章：GPU Architecture, CUDA Programming, and Maximizing Occupancy——GPU 架构、CUDA 编程与最大化“占用率”。前五章讲的是系统层面（硬件、网络、存储），从这章起全书进入 GPU 内部。这一章是第 7 到 12 章所有 kernel 级优化的**地基**，所以今天概念比较多，我会全程用大白话解释，讲一个小时左右。

第 2 页 | Roadmap: Five Parts, Each Answers One Question（路线图：五部分各答一问）

【口播】这次路线图不是目录，而是**问题清单**——表格右列就是每个部分要回答的问题：**Part I GPU 架构**（4-20 页）回答“SIMT 模型怎样把 warp/block/grid 映射到 SM 上”（中间我自己画了两张比喻图帮大家建立直觉）；**Part II CUDA 编程**（21-28）回答“如何把一个计算映射到 GPU 线程上”；**Part III 内存层级**（29-37）回答“数据放在哪里、搬动它的代价是什么”；**Part IV Occupancy 实战**（38-46）回答“更多 warp 什么时候真能提速”；**Part V 正确性与 Roofline**（47-50）回答“下一步该优化计算还是内存”。

【必说】顺带回答“章标题里的 Maximizing Occupancy 放在哪”：occupancy 就是把 Part I 的执行模型连到 Part IV-V 峰值吞吐的**工作指标**——足够多的常驻 warp 让 SM 在等数据时也有活干。

【必说】五部分的关系一句话：Part I 讲 **SIMT 硬件怎样执行**并行工作，Part II 讲**程序怎样描述**并行工作，Part III 讲**数据怎样供应**这些计算，Part IV 讲**更多并行度何时还有帮助**，Part V 讲**下一个瓶颈在哪里**。

【必说】关键一句：后面出现 SFU、TMEM、Unified Memory 这类细节页时，请大家随时问自己“它在回答哪个问题”——答案一定在这五个里。

全章一句话（金句块）：**Expose enough parallelism, keep data close, and profile the real bottleneck.**——制造足够的并行，让数据靠得近，用 profiler 找到真正的瓶颈。

第 3 页 | GPUs Optimize Throughput; CPUs Optimize Latency

【口播】CPU：核少、缓存深、单线程快。GPU：一百多个 SM 并行跑几千个线程。右图（Figure 6-1）是最基本的协作流程：host（CPU 侧）把数据拷到 GPU 显存、启动 kernel、拷回结果——注意有两次跨设备拷贝。GPU 的优势来自**大规模并行**：大量 SM 和线程同时处理不同的数据，提高整体吞吐；在每个 SM 内部，多个常驻 warp 还能填补访存和流水线停顿的空档。（注意区分：SM 内的 warp 调度隐藏的是 **HBM 访问、数据依赖、流水线停顿**；CPU↔GPU 的拷贝要靠 pinned memory + cudaMemcpyAsync + stream 与计算重叠——那是第 27、50 页的内容。）它的甜区是**数据并行**工作：矩阵乘、卷积——同一条指令作用于海量元素；可以直接写 CUDA C++，也可以经由 PyTorch 或 OpenAI Triton 这类 Python 系工具间接生成。

大白话：**CPU 是把单个零件做得快；GPU 是一座工厂——你的任务是让每个车间都忙起来。**

第 4 页 | Inside a Blackwell SM: The Resource Budget (SM 的资源账本)

【口播】先打开一个 SM，看它的资源账本。这里会先碰到一个新词 **warp**：32 个线程组成的执行小组——书里也是先用后讲，先记住这个定义，第 12 页正式展开。账本上三组数字要记住：

□ 每 SM 同时跟踪 **64 个 warp = 2048 个线程**——这是调度器来回切换的“池子”；

□ **64K 个 32 位寄存器**（共 256 KB），单个线程最多用 **255 个**；

□ **256 KB 统一的 L1/共享内存**，其中最多 **228 KB** 可配成用户管理的共享内存（实际可用 **227 KB**——CUDA 每 block 保留 1 KB）。

正是这些大额片上预算，让一个 SM 能“玩杂耍”般同时伺候几千个线程而不用频繁下 DRAM。右图（Figure 6-2）就是 SM 的内部结构，下一页细看。

大白话：**SM 是一个车间：寄存器是每人的工具腰带，共享内存是中间的工作台。**

【问】可能被问：“227 和 228 怎么回事？”→ 共享内存可配上限 228 KB，但 CUDA 每 block 保留 1 KB，所以单 block 最多申请 227 KB。

【问】可能被问：“执行到底发生在哪？kernel 上还是 warp 上？”→ 都不是——**执行发生在 SM 分区里的物理执行单元上**（图 6-2 里的绿色格子：INT32/FP32 ALU、LD/ST、SFU、Tensor Core）。kernel 是菜谱（指令文本），warp 是厨师班组（谁），执行单元是灶台（在哪）：调度器每拍安排一个班组去灶台做菜谱的下一步。顺带纠正：“CUDA core”不是 CPU 那种核，只是一条**FP32 算术车道**，没有自己的取指译码；一个 warp 执行一条 FP32 指令占 32 条车道。车道少的单元 warp 要分几拍过——SFU 每分区只有 4 条，32 个请求分 8 拍流完，这就是 SFU 吞吐低的物理原因。

第 5 页 | SFUs and Load/Store Pipelines (特殊功能单元与访存管线)

【口播】SM 里还有两类容易被忽略的部件。左边：**SFU (Special Function Unit, 特殊功能单元)**，专算超越函数——sin、cos、倒数、开方。关键点：它有**自己独立的管线**，不占核心“数学 + 访存”管线的发射名额（“双发射”机制第 7 页细讲）——慢速复杂运算永远不会堵住核心管线，混合运算的 kernel 因此有更多指令级并行。

右边：**LD/ST (load/store) 访存管线**，每 SM 共 16 条（每调度器 4 条），负责读写 L1/共享内存、L2 和全局显存。书里特别警告：**具体管线数量和配对规则不受保证**——判断 kernel 是“访存发射受限”还是“计算发射受限”要靠 profiling，细节查 Blackwell tuning guide。

大白话：**SFU 是商店后面的专柜——复杂业务去那儿办，快速通道保持流动。**

第 6 页 | Analogy: The GPU as a Factory (比喻: GPU 工厂全景图) □

【口播】一间车间看完，拉远看全厂——这页就一张图、一句话：**GPU 是一座工厂**。顺着图把两套词汇立起来（口头讲，页面上不写字）：

任务一侧：grid = **这一整批订单**；block = **一个班组**——每个班组被安排进一间车间（SM），整个生命周期不换车间，但一间车间通常**同时容纳好几个班组**；warp = 车间把班组每 32 人编成的**执行小队**；thread = **一名工人**。

数据一侧：Register = 随身工具箱，L1 = 车间临时货架，Shared Memory = 公共工作台，**L2 = 全厂中转仓，HBM = 厂外大仓库**——一次全局读取依次尝试 L1 → L2 → HBM，数据原路返回。

【必说】工具箱补一句（后面 occupancy 和 spilling 都靠它）：**工具箱不是工人自带的，是从车间那块大柜子（register file，每 SM 64K 个寄存器）里静态划走的一格——block 进驻时划走、干完才归还**。所以：常驻 warp 切换零成本（工具箱一直在车间里）；每人工具箱越大、车间站的人越少（寄存器 × 线程数 ≤ 柜子容量，occupancy 权衡的物理来源）；装不下的溢出到城另一头的自助仓储（local memory，落在 DRAM）= spilling。

【问】较真的听众可能问：“register file 是四个分区共享的吗？”→ 严格说每个分区各有一块（每分区 16K、共 64K），warp 归哪个分区、寄存器就在哪个分区的柜子里；书里只讲到“64K per SM”这个粒度，回答到这里即可。

【必说】收一句：“这张图后面每个部分都会回来用：Part I 讲车间和小队，Part III 讲中转仓和大仓库。”

【问】可能被问：“block 会不会跨车间？”→ 不会，从生到死在同一个 SM 上；但 block 和 SM 不是一一对应。

第 7 页 | Four Warp Schedulers, Dual Issue (四调度器、双发射) □

【口播】每个 SM 其实是**四个“迷你 SM”**：四个独立 warp scheduler（warp 调度器），各带自己的派发逻辑。两层机制：

□ 每个调度器每拍发射一个 warp 的指令 → **每拍最多 4 个 warp 同时推进**；

□ **dual-issue (双发射)**：同一拍里同一个 warp 可以同时发一条算术指令（INT32/FP32/Tensor Core）+ 一条访存指令（load/store）。**限制：必须来自同一个 warp，不能跨 warp 拼。**

最好情况：每拍 **4 数学 + 4 访存** 指令齐飞。右边表格（Table 6-1）是每拍的上限。注意表格下的小字：书里所有指标表的数值都是**示意值**，真实 benchmark 在配套 GitHub 仓库——这是全书的统一免责声明。

大白话：**同一层楼四名调度员，每拍各派出一支小组——而且小组可以一手计算、一手取料。**

【问】可能被问：“发射一个 warp 和 warp 正在执行计算有什么区别？”→ **发射（issue）**是调度器的动作：一拍递一条指令进管线，递完就去管别人；**执行（execute）**是功能单元的活儿：指令在管线里流若干拍才出结果（算术约 4 拍、访存几百拍）。所以“每拍最多发 4 个 warp”≠“只有 4 个 warp 在干活”——某一瞬间几十个 warp 的指令同时在各条管线里流动。车间话术：调度员每拍只递 4 张工单，但车间里几十支小队正在工位上加工半成品。图 6-7 每一行画的是一次**发射事件**，不是执行全程。进阶：下一条指令若不依赖上一条的结果，可以背靠背发射（ILP）——这就是结尾“ILP 足够时低占用率也能赢”的原理。

第 8 页 | Analogy: One SM Is a Workshop (比喻: SM 车间图) □

【口播】前面几页的数字有点抽象，我画了一张图，把整个车间装进一页。我们把一个 SM 想成一间大车间。车间里有**四个调度分区**，形象地叫 4 个 mini-SM——注意它们**不是**四个独立的 SM，CUDA 程序只能看到整个 SM，不能指定“把这个 block 放到 mini-SM 2”；mini-SM 只是帮助理解内部调度的概念。每个分区有自己的 **Warp Scheduler**，相当于一名调度员。

货架上的**安全帽**：一顶安全帽代表一个完整 warp 的执行状态（不是一个线程）。每个分区约 16 顶 $\times$ 4 个分区 = 整个 SM 最多 **64 个常驻 warp** = 2,048 线程。“常驻”的意思是寄存器、状态都已分配好、随叫随到——**不等于 64 个 warp 同一拍都在执行**。

货架下面**被高亮的工人小组**：这一拍被调度员选中（发射指令）的 warp——4 名调度员每拍最多各挑 1 个，所以图中间写“每拍最多选中 4 个 warp”。

左下角单独放大：**1 warp = 32 个线程**，SIMT 下同一条指令、32 个不同的 idx，调度员永远整组整组地派工。

图最底部是共同的地基——Register File、L1、Shared Memory——说明四个分区仍然共用同一个 SM 的片上资源。

大白话：**64 支候选队伍待命，4 名调度员，每拍最多派出 4 支——每支队伍 32 名工人。**

【问】可能被问：“mini-SM 是真实硬件吗？”→ 是真实存在的调度分区（processing block），各有独立的 warp scheduler 和执行单元，但对 CUDA 编程模型不可见、不可指定，所以说它是“理解硬件调度的概念”。

【问】“16 顶安全帽是精确值吗？”→ $64 \div 4$ 的平均示意；实际驻留分布由硬件决定。

第 9 页 | The Thread Hierarchy: Threads → Blocks → Grids (线程层级)

【口播】CUDA 把并行工作组织成三层（右图 Figure 6-3）：**thread（线程）**——处理一个数据元素的工人；**thread block（线程块，又名 CTA，协作线程阵列）**——最多 1024 线程一组，组内共享快速的片上共享内存；**grid（网格）**——一次启动的全部 block，尺寸设对可以扩展到几百万线程、kernel 一行不改。调度和分发由 CUDA 运行时（以及 PyTorch）自动完成。

大白话：**工人组成班组，班组内交流便宜；公司按活儿多少雇任意多个班组。**

第 10 页 | Blocks Cooperate Inside, Stay Independent Outside (块内协作、块间独立)

【口播】两条规则一正一反。**块内**：线程用共享内存交换数据、用 `__syncthreads()` 同步——这是个 barrier（栅栏），所有人到齐才继续。但**每个 barrier 都有开销**，书里明确说：**尽量减少同步点**（右图 Figure 6-6）。

块间：完全独立、执行顺序不保证任何东西。这个“不方便”恰恰是 CUDA 可扩展性的来源——调度器可以把 block 随意撒到所有 SM；你的代码在未来 SM 更多的 GPU 上**不改就能跑**。

大白话：**班组碰头会（barrier）有用但贵，能少开就少开；而且永远别假设 A 班组比 B 班组先干完。**

【问】可能被问：“barrier 让 block 等待，和 warp 切换有什么关系？”→ 在硬件眼里，“在 barrier 等队友”和“等全局内存/等数据依赖”是同一种状态：这个 warp 暂时**没就绪**。调度器的反应永远一样——跳过它、发射别的就绪 warp（同 block 的，或同 SM 上其他 block 的；barrier 只拦自己 block）。注意：切换救的是**机器利用率**，不是你的 block 耗时——block 还是要等最慢的 warp 过线，所以“少同步”的忠告依然成立。

第 11 页 | Thread Block Clusters and DSMEM (线程块簇与分布式共享内存)

【必说】本页非本期重点，30 秒带过，标了星号。

【口播】传统上不同 block 的线程不能直接协作，现代 GPU 打破了这一点：**thread block cluster (线程块簇)**——一组能跨 SM 通信的 block，有簇级硬件 barrier。底层是 **DSMEM (分布式共享内存)**：把参与簇的各 SM 的共享内存 bank 用片上高速互连连成一个池子（右图 Figure 6-5）。效果：不同 block 的线程能以片上速度读、写、原子更新彼此的共享缓冲——**不花全局显存带宽**。这是今天大矩阵乘、LLM 负载的关键使能技术，第 10 章细讲，今天知道它存在即可。大白话：**相邻班组在工作台之间的墙上开了个门，零件直接递过去，不用再走仓库。**

第 12 页 | Warps and SIMT: 32 Threads in Lockstep

【口播】上一页的三层是软件视角；硬件看到的还有一层：block 会被再切成 **warp**，**固定 32 个线程一束**，在 **SIMT** (single instruction, multiple threads, 单指令多线程) 模型下**锁步 (lockstep)** 执行——32 个人同一拍做同一个动作。三个要点：□ **硬件真正调度的单位是 warp，不是单个线程**——这是理解 GPU 的关键一跳；□ warp 的大小**每一代 GPU 都是 32**，这个数可以焊死在脑子里；□ 硬件靠**快速切换 warp** 来隐藏长延迟事件（全局加载、缓存填充、管线停顿）。

大白话：**warp 是 32 人的施工小组——同一条指令全组一起动，否则整组等着。**

有了 warp 这个词，我们就可以打开 SM 看内部了——接下来三页全在 SM 里面。

第 13 页 | Why SIMT Matters: One Instruction Drives 32 Lanes (SIMT 的意义) □

【口播】上一页说 warp 32 人齐步走，这一页回答“为什么要这样设计”。看图 6-7: warp scheduler 每拍发**一条指令**，整个 warp 的 **32 条通道**同时执行它，各自处理**不同的数据元素**——图里每一行就是“一条指令、整组推进”。

【必说】**读图法 (重要)**：注意左列的指令编号——Warp 8 的 instruction 11 和 instruction 12 **不是背靠背的**，中间插着 warp 2、warp 14 的指令。为什么 warp 8 歇了几拍？因为它的下一条指令还没就绪（在等内存、等依赖、或在 barrier 处等队友）。**这些“插队”就是延迟隐藏本身**：一个 warp 等待的空拍，被别的 warp 的工作填满。图里没标停顿原因，所以要靠编号读出来——第 16 页会把这个过程逐拍摆开。

好处：取指、译码、调度这些控制成本，**一条指令摊给 32 个线程**——控制逻辑做小了，省下的硅片面积全部给了算术单元。这就是 GPU 能塞下几千个“核”而 CPU 不能的根本原因。

也因此，**硬件调度的最小单位是 warp 而不是线程**：线程从不单独前进，整个小队一起走。

代价：32 人绑在一起，想走**不同的路**怎么办？——下一页。

大白话：**一张工单同时指挥 32 名工人——管理便宜；大家意见一致时威力大，意见不合时就尴尬。**

第 14 页 | Warp Divergence: When if/else Splits the Crew (warp 分支分歧) □

【口播】尴尬的情况就是 **warp divergence (warp 分支分歧)**。看左边代码：if (i % 2 == 0) 做任务 A; else 做任务 B;。同一个 warp 里：**16 个线程想走 A, 16 个想走 B**。但它们原则上必须一起执行同一条指令，所以 GPU 只能：**先执行 A、把走 B 的 16 个线程屏蔽掉；再执行 B、把走 A 的 16 个屏蔽掉**——总时间 $\approx$ 两条路径之和，效率直接掉一半；分支路径越多，串行化越狠。图 6-8 就是这个对比：左边分叉（红蓝交错、走走停停），右边整齐一致（全速前进）。

两个要点：□ **跨 warp 没有惩罚**——不同 warp 各走各的分支完全免费；□ 修法：让分支条件按 **warp 对齐** (i / 32, 一整队都走同一边)，而不是按奇偶 (i % 2, 队内劈两半)。

检测靠 profiler (第 8 章 Nsight Compute)。

大白话：同一小组一半人接了 A 工单、一半接了 B 工单，全组就得把两份活各干一遍。

第 15 页 | Worked Example: One Million Adds Through Both Views (实例：一百万次加法) □

【口播】Part I 收尾，用一个例子全部串起来。先垫一句：**这个三尖括号语法是 Part II 的正题，这里先当“读法”看**——尖括号里第一个数是 block 数、第二个是每 block 线程数。左栏是你写的**订单**：add<<<3907, 256>>>——1 个 grid; **3,907 个 block** (一百万除以 256 向上取整)；每 block **256 线程**——到这里都是你选的；硬件接手把每个 block 切成 **8 个 warp**。每个线程只说一句话：“我处理第 idx 个元素。”

右栏回答“**为什么是 256**” (best practice 的完整理由)：□ **32 的倍数**——恰好 8 个整 warp，没有半空 warp 白占调度位；□ **离 1,024 上限足够远**——每线程寄存器够用、block 的共享内存份额不大；□ **够小、能叠**——一个 SM 能同时放好几个 256 线程的 block → 常驻 warp 多 → 延迟隐藏有本钱；□ **不是魔法数**——128-512 都合理，最终靠 profiling 定。

【必说】口头补充硬件怎么消化这 3,907 个 block (页面上不写)：它们**不会同时进 GPU**——假设 4 个 SM、每个放 2 个 block：先进 8 个，每 SM 16 个常驻 warp；做完一个，队列里的下一个顶进来，一批批做完 3,907 个。GPU 有 4 个 SM 还是 400 个，**同一份代码**——grid 把程序和 GPU 规模解耦。

大白话：**你写的是订单 (3,907 个 256 人的班组)**；硬件决定哪些班组进哪间车间，并把每个班组编成 **32 人的执行小队**。

【问】“ $3,907 \times 256 = 1,000,192 > \text{一百万}$ ，多出来的 192 个线程呢？”→ 边界检查 if (idx < N) 的意义 (第 25 页细讲)。

第 16 页 | Let the Workshop Move (让车间动起来) □

【口播】先定位这一页：前面讲的是并行的**第一层收益**——很多线程同时计算；这一页讲**第二层收益**——某个 warp 暂时动不了时，其他 warp 怎么让 SM 保持忙碌。这才是 latency hiding 的正式出场。把第 8 页那张车间图**动起来**：假设某个分区此刻：Warp A 在等 HBM 数据、Warp B 在等上一条计算结果、Warp C 和 D 数据都齐了。调度员**不会**傻等 A 和 B——它直接发射就绪的 C 或 D，跳过停顿的 warp 不花任何代价。下一拍它可能又挑别的：周期 1 发 C、周期 2 发 F、周期 3 发 D、周期 4 发 H……

两个精确的说法：□“选中”更准确说是**发射 (issue) 指令**——指令开始执行，不代表这一拍执行完；□延迟隐藏**不是**让某个 warp 的等待消失，而是**它等待时让别的 warp 干活**，把等待“填满”。这就把第 4 页的 64 这个数字讲活了：64 个常驻 warp 的意义，就是给 4 个调度器准备足够深的**就绪候选池**——反过来，占用率低 = 池子浅 = 没得可切 = 等待变成纯空转。这句话记住，第 39 页起的 22 倍案例就是它的实证。

大白话 (v2 收束句，只强调这一句)：**延迟隐藏不是缩短等待，而是用别的小组的活儿把等待填满。**

第 17 页 | Occupancy Gives the Scheduler More Choices (occupancy = 给调度员更多选择) □

【必说】v2 必说 (讲完定义马上补，防止第 52 页“反转”显得突然)：“occupancy 不是 GPU 有多忙，也不是性能分数。它只说明 SM 上有多少 resident warps 可供 scheduler 挑选。它提高的是调度器**找到 ready warp 的概率**——不保证一定存在 ready warp，也不保证性能一定更高。足够多可以隐藏延迟；达到足够之后，继续提高可能没有收益。”

【口播】现在正式定义标题里的概念。**Occupancy = SM 上活跃 warp 数 ÷ 硬件上限 (Blackwell 是 64)**。profiler 里实测的平均值叫 **achieved occupancy**。

为什么要高？**latency hiding (延迟隐藏)**——一个 warp 卡在访存上，另一个随时顶上。Blackwell 的大寄存器堆 (64K/SM) 让高占用率更容易达到。

但立刻要说“制衡”：warp 们**共享寄存器和共享内存**。塞太多 warp → 每线程分到的寄存器变少 → **register spilling (寄存器溢出)** 到慢速内存——你亲手制造了新的停顿。所以书里的忠告是：**把 occupancy 和寄存器/共享内存用量放在一起 profile。**

大白话：**occupancy 不是“GPU 有多忙”，更不是性能分——它只是调度员手里有多少支常驻候选队伍：够用就能藏延迟，超过够用再堆未必有收益 (第 52 页会呼应这一点)。**

第 18 页 | Registers and Occupancy: A Worked Example (寄存器 × 占用率算例) □

【提示】上一页刚说“占用率要和寄存器/共享内存放在一起看”，这页把这笔账真正算一遍。

【口播】寄存器就是线程手边放计算数据的小格子。一个 32 位格子（4 字节）能放一个 float 或 int；double 要两个格子；两个 FP16 可以打包放进一个格子（half2）。整个 SM 的格子总量固定（本例 65,536 个），但**每个线程保存自己的数值**。左下代码就是这个意思：数组 A、B、C 住在显存里，参与计算的 a、b、c 待在寄存器里（示意代码，真实分配由编译器决定）。

看右边的表：每个 block 有 256 个线程 = 8 个 warp。如果每线程用 **64 个寄存器**，一个 block 就用掉 **16,384** 个，整个 SM 只能同时放下 $65,536 \div 16,384 = 4$ 个 block，也就是 **32 个 warp**。硬件上限是 64 个 warp，所以 **occupancy = $32 \div 64 = 50\%$** 。上下两行同理：每线程 32 个 → 8 个 block → 100%；128 个 → 2 个 block → 25%。

关键机制：某个 warp 等待访存时，**中间结果还要保留**，寄存器不能直接让给别人。因此每个线程占得越多，同时能驻留的线程就越少，多出来的 block 通常在外面排队等。反过来，如果为了提高 occupancy **强行压低寄存器配额**，才可能增加 spilling（Part III 第 31 页的悬崖）——最终还是要看实测运行速度。

（页面小字：这是简化的容量算术，忽略了其他上限和分配粒度。）

大白话：**每人工具箱越大，车间里能站的人越少；打盹的小队工具箱也不能收走——里面还放着一半的活。**

【问】可能被问：“怎么知道我的 kernel 每线程用了多少寄存器？”→ 编译时 `nvcc -Xptxas -v` 打印，运行时 `ncu` 看 Registers Per Thread。

第 19 页 | Hardware Limits I: Warp and Block (硬件上限之一)

【必说】v2 开场定调（对应页面顶部灰字）：“这两页的数字不需要记住。我们只需要理解一件事：每个 block 占用的 threads、registers 和 shared memory，决定一个 SM 还能同时放下多少 block 和 warp。”讲完 256 与 1024 线程 block 的例子即可，不逐行念表。

【口播】查表页，重点讲右边的框：**为什么必须是 32 的倍数——一个 33 线程的 block 要占两个 warp 槽位**，第二个 warp 只有 1/32 的车道干活，却照样占一个完整的调度器名额。每个“不是 32 倍数”的选择都在给虚空捐算力。

表格三行：warp 固定 32；每 block 最多 **1024 线程**（三个维度乘起来 ≤ 1024 ）；也就是每 block 最多 32 个 warp。

下面两条补充：block 太大 → 寄存器要得太多 → **溢出**；共享内存也是 **SM 上全部常驻 block 共享 227 KB**。反过来，**block 小一点往往 occupancy 更高**——每 SM 能塞更多独立的 block。

第 20 页 | Hardware Limits II: SM-Resident and Grid (硬件上限之二)

【口播】右边图 6-9 先讲：它把线程世界的**尺度阶梯**画成一张图，从里到外一路“ $\times 32$ 、 $\times 32$ 、 $\times 2$ ”：**1 个线程 $\rightarrow \times 32 = 1$ 个 warp $\rightarrow \times 32 =$ 单 block 上限 (1,024) $\rightarrow \times 2 =$ 单 SM 常驻上限 (2,048) $\rightarrow$ grid 基本无上限 (X 维约 21 亿个 block)**。这页你只需要记住这串乘法，表格都是它的展开。工厂话术：一名工人 $\rightarrow$ 32 人小队 $\rightarrow$ 一个班组最多 32 支小队 $\rightarrow$ 一间车间同时在册 64 支小队 $\rightarrow$ 订单簿想开多长开多长。

左表是每 SM 的常驻 (resident) 上限：**64 warp、2048 线程、32 block**——64 这个数已经**好几代不变**，所以 occupancy 的经验能跨代延续。

算一笔账 (这页最有用的部分)：每 block 1024 线程 $\rightarrow$ 一个 SM 只能驻 **2 个 block**；改成 256 线程 $\rightarrow$ 能驻 **8 个**——同样 2048 线程，灵活性完全不同。

grid 上限：X 维约 21 亿个 block，Y/Z 各 65,535；每设备最多 **128 个 kernel 并发**。实践结论：**你永远先撞上 per-SM 限制**；真要超过 Y/Z 上限，用 2D/3D grid 或分多次启动 (multilaunch)。

第 21 页 | Compatibility: PTX, SASS, and Fatbins (兼容性模型)

【口播】CUDA 生态的护城河之一：**前后向兼容**。三个概念：**SASS**——特定架构的最终机器码 (sm_90 = Hopper, sm_100 = Blackwell)，只带单架构 SASS 的二进制上不了新 GPU；**PTX**——虚拟指令集，驱动在加载时 JIT 即时编译成新架构的 SASS，这就是**前向兼容**的机制；带 f 的家族目标 (如 sm_100f) 只在同特性家族内可移植。

最佳实践 (右框)：发 **fatbin (胖二进制)** ——通用 PTX + 需要的家族专用 cubin，并为其他架构留 fallback。验证方法：设 `CUDA_FORCE_PTX_JIT=1` 强制走 PTX JIT——**二进制里没有 PTX 的话 kernel 启动直接失败**，逼你重新构建。

大白话 (工程结论)：**性能代码不仅要跑得快，还要能跨 GPU 代际运行——所以发布时通常同时包含优化后的 SASS 和用于 forward compatibility 的 PTX。**

【必说】讲法建议 (40 秒版，不用讲上面的细节)：开场用报错当钩子——“如果你见过 no kernel image is available for execution on the device 这个报错，讲的就是这页的事。”然后只说三句：□ 编译产物有两种：SASS 是给某一代卡的最终机器码，PTX 是虚拟汇编、驱动加载时现场编译；□ 只带 SASS 的程序换新一代卡就是上面那个报错；□ 所以发布打 fatbin：优化好的 SASS + 兜底的 PTX。家族目标、`CUDA_FORCE_PTX_JIT` 这些留在页面上当参考，口头不展开；有人追问再说。

第 22 页 | Anatomy of a CUDA Kernel: Device Side (kernel 解剖：设备侧)

□

【口播】这是全书第一个完整的 kernel，五行，逐行看：

第 1 行 `**__global__ void myKernel(float* input, int N)**`——`__global__` 修饰符的意思是：这个函数在 GPU（设备）上运行，但从 CPU（主机）上调用。参数就是一个裸指针加一个长度。

第 2 行 `**int idx = blockIdx.x * blockDim.x + threadIdx.x;**`——全页最重要的一行。三个内置变量：`blockIdx`（我在第几个 block）、`blockDim`（每个 block 多大）、`threadIdx`（我在 block 内排第几）。三者一乘一加，每个线程算出一个全局唯一的编号——这就是它“领到的那一格数据”。

第 3-4 行 `**if (idx < N) { input[idx] *= 2.0f; }**`——边界检查（第 25 页专门讲为什么），然后就地把自己那个元素翻倍。

右侧三个要点：□ kernel 是站在单个线程的视角写的，`idx` 划出属于这个线程的那一小片数据；□ CUDA 把它编译成设备代码，由成千上万乃至上百万个轻量线程并行执行——你写一份，它跑一百万份；□ 主机侧的启动语法 `**<<<blocksPerGrid, threadsPerBlock>>>**`——每一次 kernel 调用都绕不开的两个核心参数（Part I 实例页已经见过了）。

大白话：你只写一名工人的岗位说明书；CUDA 复印一百万份，每份填上不同的行号。

【问】可能被问：“这是什么语言？CUDA 是一门新语言吗？”→ 是 **CUDA C++**：主体就是标准 C++，扩展只有几样——`__global__` 这类修饰符、`<<< >>>` 启动语法、`blockIdx` 等内置变量，外加 `cuda*` 运行时 API（这是函数库不是语法）。编译器 `nvcc` 把一份 `.cu` 劈成两半：主机代码走普通 C++ 编译器，kernel 编成 PTX/SASS（呼应 Part I 兼容性页）。会 C++ 就会 90%。

第 23 页 | Host Side: The Six-Step Data Flow (主机侧：六步数据流)

【口播】kernel 写好了，主机侧要干六件事把它伺候起来。左边代码从上往下：

第 1 步：分配。`cudaMallocHost(&h_input, ...)` 在主机上分配锁页（pinned）内存——注意不是普通 `malloc`；然后初始化数据；`cudaMalloc(&d_input, ...)` 在 GPU 显存上分配同样大小。命名习惯全书通用：`**h_` 前缀 = host 指针，`d_` 前缀 = device 指针 `**`——两个地址空间，千万别拿错。

第 2 步：拷过去。`cudaMemcpy(d_input, h_input, ..., cudaMemcpyHostToDevice)`——H2D。

第 3 步：启动。算好 `threadsPerBlock = 256`、`blocksPerGrid = (N + 255) / 256`（一百万元素就是 3,907 个 block），然后 `myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input, N)`。

第 4 步：等。`cudaDeviceSynchronize()`——kernel 是异步启动的，主机要显式等 GPU 干完。

第 5 步：拷回来。`cudaMemcpy(..., cudaMemcpyDeviceToHost)`——D2H。

第 6 步：清理。`cudaFree + cudaFreeHost` 配对释放。

右侧两个要点：□ 为什么用 `cudaMallocHost` 而不是 `malloc`——锁页内存是后面异步拷贝真正实现“传输与计算重叠”的先决条件（第 28 页和 Part V 会兑现这个伏笔）；□ 书里自己说明：这个版本还没优化——它是一份简单而完整的模板，全书后面的章节都在改进它。

【问】可能被问：“锁页是什么意思？”→ 操作系统承诺这块内存不会被换页/挪动，物理地址固定，GPU 的 DMA 引擎才能绕过 CPU 直接搬运——这是异步拷贝的硬件前提。代价是占住物理内存，所以别把所有东西都 pinned。

第 24 页 | Why Pass N? (为什么要传 N)

【口播】一个初学者必问的问题：数组多长你自己不知道吗，为什么非要把 N 传进 kernel？蓝框是书里的回答，意译：**CUDA kernel 被设计为在单个线程内部工作，与成千上万个其他线程并肩，各自处理输入数据的一个分片——N 定义了这个分片的边界。**

展开成三点：□ CPU 函数可以自己查容器的长度（vector 有 size()）；kernel **做不到**——它拿到的只是一个**裸指针**，指针不携带长度信息，必须有人告诉它“世界的边界在哪”；□ N 与 blockIdx/blockDim/threadIdx 配合，让一百万个元素被**不重、不漏、干净地**并行处理，横跨多个 SM；□ 这就是从 CPU 到 GPU 编程**最核心的思维转变**：你写的不是“处理这个数组”（一个循环），而是“处理**属于我的**那个元素——如果它存在的话”（一个判断）。

大白话：**一百万份复印的岗位说明书内容全一样；N 是上面写“活儿到哪里为止”的那一行。**

第 25 页 | The Bounds Check –and Lazily Surfacing Errors (边界检查与懒惰报错) □

【口播】上一页说“如果它存在的话”，这页把这件事彻底讲清，分四步。

第一步：多出来的线程从哪来。蓝框的启动配置：myKernel<<<1, 64>>>(input, 63)——数组 63 个元素（下标 **0–62**），但 GPU 启动了 64 个线程（threadIdx.x = 0–63），组成 **2 个 warp**（注意 CUDA 从 **Warp 0** 开始编号）：**Warp 0** 管线程 0–31，全部有效；**Warp 1** 管线程 32–63——其中 32–62 有效，**线程 63 就是超出有效范围的那一个**。对它来说 idx = 63、N = 63，63 < 63 为 false——它不碰数组。

第二步：这个 if 会不会让整个 warp 都不执行？不会。Warp 1 里 lane 0–30 (idx = 32–62) 照常做乘法，只有 **lane 31 被 active mask 屏蔽**。一个 warp 按同一条指令执行，GPU 用活跃掩码控制哪些通道真正写数据——这里确实有一点分支分化，但只发生在最后一个 warp，通常可忽略（呼应 Part I 的分歧页）。

第三步：不写检查会怎样？是未定义行为，不是“必然报错”。线程 63 会执行 input[63] *= 2.0f，而最后一个合法元素是 input[62]。后果三选一：**可能报 cudaErrorIllegalAddress；可能暂时什么错都不报；甚至可能悄悄破坏别的数据**——因为 input[63] 虽然超出数组的逻辑范围，却可能仍落在 GPU **已映射的内存页**里，硬件只判断“这个地址不可访问”，它不知道“这个数组只到 62”。这就是越界 bug 难排查的原因：有时能跑，换个数据大小就炸。

第四步：为什么 CPU 不会立刻知道。kernel launch 默认**异步**：CPU 提交启动命令后不等 GPU 干完就继续往下跑；真正的非法访问发生在 GPU 执行期间，而 CPU 要到**下一次同步或某个 CUDA API 调用**时才看到错误——这就是标题里的 **Lazily Surfacing (延迟暴露)**。看右边的时间线代码：错误真正发生在 launch 那行，却在 cudaDeviceSynchronize() 那行才被报告——**最容易冤枉人的地方**：看起来是 sync 出错了，其实它只是替前面的异步 kernel“收到了错误通知”。实践对策：调试期在每次 launch 后紧跟 cudaGetLastError() + cudaDeviceSynchronize()，把错误按在案发现场。

大白话：**GPU 出事不会给你打电话——它留张字条，你下次开信箱才看到。**

【问】可能被问：“生产代码也要每个 launch 都加 sync 吗？”→ 不要——cudaDeviceSynchronize 会杀性能。调试时加；生产上靠 CUDA_LAUNCH_BLOCKING=1 环境变量临时复现，或用 compute-sanitizer 的 memcheck（书 p259 有专门一节，本次不展开）。

【问】“既然硬件不查数组边界，谁能查？”→ compute-sanitizer 的 memcheck 工具——它以调试模式跑 kernel，逐访问核对合法性，越界必抓（代价是慢，只在排查时用）。

第 26 页 | Configuring Launch Parameters: The Recipe in Code (launch 参数：代码配方)

【口播】“为什么 256”在 Part I 的实例页已经讲透，这页只兑现代码，**30 秒**：`threadsPerBlock = 256; blocksPerGrid = (N + 255) / 256`——整数除法配上“+255”就是**向上取整**，保证最后不满的一段也有 block 覆盖。技巧可推广：任意 block 大小 B，写 `**(N + B - 1) / B`。**最后一个 block 可能不满员——这正是第 25 页边界检查存在的原因，前后呼应一下。书里对 Blackwell 的提示：256-512 都可考虑；把 256 当起点 ** 而不是答案，正式跑之前用 profiling 对比 128/512。**

大白话：**256 是 block 尺寸里的“中杯咖啡”——第一次点单几乎不会错，profiling 说要改再改。**

第 27 页 | 2D and 3D Kernel Inputs (二维与三维输入)

【口播】前面全是一维数组，处理图像、矩阵怎么办？代码只多三处：□ 索引算**两次**——x 用 .x 系变量、y 用 .y 系变量，各自“`blockIdx × blockDim + threadIdx`”；□ 边界检查变**二维**——`if (x < width && y < height)`；□ 访问前**拍平**——`idx = y * width + x`，二维坐标折回一维内存（显存里本来就是一维的）。

主机侧用 ****dim3**** 类型：`dim3 threads(16,16)`—— $16 \times 16 =$ 还是 256 个线程，只是排成方块；`dim3 blocks((W+15)/16, (H+15)/16)`——两个方向各自向上取整。

右侧要点：□ 天然适合**图像**：比如 $1,024 \times 1,024$ 的矩阵用 16×16 的 block 铺满；□ 同一套模式用 `dim3(x, y, z)` 推广到 **3D**——体数据（视频、医学影像）直接映射到线程层级；□ 书中注：本书大多数场景用 **1D 或 2D（分块）** 启动；纯 1D 时**普通 int 比 dim3 顺手**，不必为了形式统一硬上 dim3。

大白话：**同一份菜谱，两个坐标轴：每个工人领到的是（行，列）工牌，而不是单个行号。**

【必说】**主动讲一下 4 维及更高维 (batch, channel, height, width) 怎么办**：CUDA 的 grid/block 最多只有 3 维，所以 ≥ 4 维张量横竖都要线性化——**逐元素操作的标准做法就是全部拍平当 1D**：`total = N×C×H×W`，一个线程管一个元素，线程根本不需要关心自己是哪个 batch（真要坐标就用除法/取模从 idx 反解）。那为什么还留着 2D/3D？两个理由：□ 对图像/体数据**写法更自然**；□ 更重要的是 **2D block 买到了空间局部性**——卷积、stencil、转置这类要访问**邻居**的操作， 16×16 的 block 对应图像上一个 16×16 邻域，可以整块搬进共享内存复用（tiling）；拍平之后“邻居”就散了。准则一句话：**逐元素 → 拍平；要访问邻域 → 2D/3D。**

再补一个实战细节：结构化操作（batched matmul、attention）里，工程上经常拿 **grid 的 y/z 维来编码 batch 和 head**——比如 FlashAttention 的 grid 就是（M 方向块数，head 数，batch 数）。这时 3D grid 的作用是“给每个 (batch, head) 发一个独立班组”的便捷索引，不是空间含义。

【问】可能被问：“为什么是 16×16 不是 256×1 ？”→ 对二维数据，方形 block 让相邻线程访问相邻像素，访存局部性更好（第 7 章 coalescing 细讲）；对一维数据 256×1 就是标准答案。

第 28 页 | Allocate Asynchronously: Streams and Memory Pools (异步分配：流与内存池) □

【口播】到目前为止我们用的 `cudaMalloc/cudaFree` 有个隐藏的大坑，右栏第一条：它们是**同步且昂贵的**——每次调用要做**全设备同步**（等 GPU 上所有活干完!），还要走操作系统调用（`mmap/ioctl`）、陷入内核态再切回来。训练循环里每个迭代分配释放几次，这个开销就把你拖死了。

解法是左边六行代码：□ `cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking)` 建一条**非阻塞流**——“流”就是一条 GPU 命令队列，同一条流内按序执行，流与流之间可以并行；**非阻塞**是为了避开老式**默认流的隐式全局栅栏**（书中提示；多流重叠第 11 章展开）；□ `**cudaMallocAsync(&d_buf, size, s)`——从每设备的内存池 `**` 里取一块，不走操作系统；□ `kernel` 照常发进流 `s`；□ `**cudaFreeAsync(d_buf, s)`——把“释放”作为一个命令排进流里，只等流 `s` 自己干完 `**`，不做全局 `cudaDeviceSynchronize`、不跨流同步。

内存池的三个好处（右栏第二条）：缓冲区**循环利用**、**没有操作系统往返**、长训练循环下**碎片更少**。

大白话：**在停车场养一支车队、用时拿钥匙——别每次送货都买车再卖车。**

【问】可能被问：“流（stream）到底是什么？”→ 一条 GPU 命令队列：队内按序、队间可并行。你之前没写过流也一直在用——不指定就进“默认流”。

第 29 页 | Tuning the Pool; PyTorch's Caching Allocator(池调优与 PyTorch 分配器) ——60 秒预告页

【口播】先定位：**这页是预告，不是深潜**——内存池的完整展开在第 11 章（stream-ordered allocation 有一整章），而且第 12 章（我下次讲）还会在 CUDA Graphs 里再遇到它。今天只要一个比喻加一个钩子。

比喻（把上一页的池讲透）：内存池就是**食堂的餐盘架**。没有池：每顿饭买个新盘子、吃完扔掉——每次分配释放都惊动操作系统。有池：盘子洗洗放回架子，下一个人直接拿——快，也不会越用越碎。左框两个旋钮就是食堂经理的两个决定：**架子上囤多少盘子才开始处理掉一些**（`cudaMemPoolAttrReleaseThreshold`：囤得多取用快、但占地地方）；**现在立刻清一批**（`cudaMemPoolTrimTo`）。权衡就一句：**总占用 vs 碎片**。

钩子（右框，大家最有感的部分）：**PyTorch 就是一个自带餐盘架的食堂**——你每次建 tensor，它从自己的缓存分配器拿，根本不调 `cudaMalloc`。三个日常现象一次解释清：`nvidia-smi` 显存为什么比模型实际用量大（架子上囤着空盘子）；OOM 报错里 `reserved` 为什么远大于 `allocated`（同理）；`PYTORCH_ALLOC_CONF` 是干嘛的（给食堂经理下指令）。

底部结论：一次性大缓冲（如模型权重）用阻塞的 `cudaMalloc` 没问题；高频周转的循环（训练！）用异步 + 池，更稳更快。

收尾转场：分配的事到此为止，细节留给第 11 章。下一个问题——分配出来的内存**住在哪、访问代价是什么**？这就是 Part III。

大白话：**** 内存池 = 食堂餐盘架；PyTorch = 自带餐盘架的食堂——nvidia-smi 里“虚高”的显存就是架子上囤着的空盘子。 ****

【问】可能被问：“碎片化怎么治？”→ PyTorch 侧可试 `PYTORCH_ALLOC_CONF=expandable_segments:True`（可扩展段，显著缓解长训练的碎片 OOM）；CUDA 侧靠池本身的复用 + `TrimTo`。

第 30 页 | The Memory Ladder at a Glance (内存阶梯总览) □ 建议讲 2 分钟

【必说】转场 Part II→III (对应页面顶部灰字): “我们已经知道怎样产生足够多的线程。但它们大部分时间究竟在等什么? 通常是数据。”回忆 Part I 的工厂图——这一部分就是逐层打开它的右半边。

【口播】先看全景表(表 6-5), 一行一行往下, 规律只有一条: **越往下容量越大、速度越慢**——每一级都在拿容量换延迟和带宽。带大家扫一遍:

寄存器——线程私有, 每 SM 共 256 KB, 约 1 拍, 几十 TB/s: 最快的一层, Part I 讲过“从大柜子里划格子”。

共享内存 + L1——SM 级, 可用 228 KB, 20-30 拍, TB 级带宽。

TMEM——SM 级, 256 KB, Tensor Core 专用, 约 10 拍。

常量缓存——64 KB 空间配 8 KB 缓存, 命中且全 warp 同址时**一拍广播**。

L2——全 GPU 共享, **126 MB**, 约 200 拍, 数 TB/s。

本地内存(溢出区)——名字骗人: 线程私有没错, 但**落在 DRAM 上**, ≤ 1000 拍——寄存器装不下的东西掉到这里。

全局 HBM3e——B200 上 **180 GB、约 8 TB/s**, 但延迟也是几百到 1000+ 拍。

右图(图 6-10) **不是表的重复, 分工不同**: 左表是“参数表”——每层**多大多快**; 右图是“平面图”——每层**在哪、怎么连**, 而且画了表里没有的三样东西: 另一块 GPU (NVLink 相连)、PCIe、底部的 **CPU host memory**。讲图时用手指三个细节: □ **SMEM 和 L1 画在同一个框里、中间一条虚线**——第 32 页“一块 SRAM 两份工作”的视觉预告, 隔板就是那条虚线; □ **TMEM 单独一个框**——第 33 页的主角先混脸熟; □ **PCIe 连向 CPU host memory 的那条线**——第 37-38 页统一内存的页迁移走的就是它, NVLink 则是多 GPU 章节的伏笔。

一句话收拢: “左表告诉你每层楼多大多快, 右图告诉你楼盖在哪、楼间的路怎么修。”这页的任务只是建立地图, 后面 8 页逐层展开。

大白话: **随身工具箱 → 公共工作台 → 全厂中转仓 → 远处大仓库。活儿尽量在自己工位上干。**

【问】可能被问: “DRAM/SRAM 到底是什么?” → 先拆词: **Dynamic Random Access Memory**, 动态随机存取存储器——**Random Access (随机存取)** 指可以直接跳到任意地址读写(不像磁带要从头卷到尾); **Dynamic (动态)** 指每个 bit 存在微型电容里、会漏电、必须每秒刷新几千次。两种存储技术对照: **SRAM (静态)** 用 6 晶体管触发器存位, 极快但占面积、贵——**片上的一切**(寄存器、shared/L1、TMEM、L2) 都是它; **DRAM (动态)** 用电容存电荷, 会漏电所以要不不停刷新(“动态”由此得名), 密度高、便宜——**片外的一切**(HBM3e、CPU 内存条、local memory 的落点) 都是它。HBM 就是把 DRAM die 立体堆叠贴在 GPU 旁边。图 6-10 里 L2 下面那个“DRAM”框 = 本卡的 HBM。整个阶梯的分界线就一条: 片上 SRAM / 片外 DRAM。

【问】可能被问: “为什么不把所有内存都做成寄存器那么快?” → 物理规律: 越快的存储(SRAM 触发器)单位容量越占面积、越耗电。8 TB/s 的 HBM 已经是堆了十几层 DRAM die 的结果。阶梯不是设计缺陷, 是物理约束下的最优解——软件的任务就是把热数据留在上层。

第 31 页 | Register Pressure Can Turn Fast Storage into DRAM Access (寄存器与溢出悬崖) □

【口播】标题就是结论：**寄存器压力能把最快的存储变成 DRAM 访问**。四个要点：

□ 每个线程的数据之旅都从**寄存器堆**出发：单拍读写、几乎不与任何人抢，每 SM 几十 TB/s——回忆 Part I：工具箱是从车间大柜子里划走的一格。

□ 预算：每 SM 64K 个 32 位寄存器，**单线程最多 255 个**。

□ **悬崖在这**：局部变量太多、编译器临时量太多，超出的部分**溢出 (spill) 到 local memory**——名字里带“本地”，实际住在**片外 DRAM**：从 1 拍直接掉到几百上千拍，这就是“悬崖”两个字的意思。右图（图 6-12）画的就是这个溢出区。

【必说】**词义先讲一下 (spill)**：spill 本义是“（液体）洒出来”——杯子满了，多的水洒到桌上。这里的“杯子”是每线程最多 255 个寄存器；装不下的变量被编译器“洒”到 local memory。工厂话术：**工具箱装满了，多出来的工具寄存到城另一头的自助仓库，每次用都得跑一趟**。一句话记住它和 DRAM 的关系：**spill = 数据从片上 SRAM（工具箱）被挤到片外 DRAM（城外仓库）——这就是它是性能杀手的原因**。

□ 怎么防：盯住 Nsight Compute 里的 **“Registers Per Thread”** 指标——**溢出是无声的杀手**，代码不报错、结果全对，只是莫名其妙地慢。

和 Part I 的呼应：__launch_bounds__（书 p257 的调优工具，本次不展开）就是主动限制每线程寄存器数来换占用率的工具——那是硬币的另一面。

大白话：**“local memory”是 CUDA 里最误导人的名字——它是城另一头的自助仓储，不是你的口袋**。

【问】可能被问：“怎么知道自己 spill 了？”→ 两个办法：编译时 nvcc -Xptxas -v 会打印每 kernel 的寄存器数和 spill stores/loads 字节数；运行时 ncu 看 Registers Per Thread 和 local memory 流量。看到 spill 字节数 > 0 就要警惕。

第 32 页 | Shared Memory + L1: One SRAM, Two Jobs (共享内存与 L1: 一块 SRAM 两份工作) □

【口播】这页的大意一句话：每个 SM 有一块速度很快的片上 SRAM (256 KB)，它既可以当硬件自动管理的 L1 Cache，也可以当程序员管理的 Shared Memory (最多 228 KB)。Shared Memory 让同一个 block 的线程共享和复用数据，20-30 拍、TB 级带宽。

重点讲蓝框：Shared Memory 为什么能加速——装一次、用很多次。以矩阵乘法为例：一个 block 负责算一个输出 tile。不用 Shared Memory 时，很多线程会反复从 Global Memory 读同一段数据——线程 0 读 A 的一段，线程 1 又读同一段，线程 2 又读一遍……全是几百拍的重复搬运。用上之后，数据流变成：Global Memory → (每份数据只装载一次) → Shared Memory → (block 内所有线程反复复用) → 计算。同一段 A 从“人人下仓库”变成“一人取回、全组共用”，全局内存流量直接除以复用次数。

两个代价 (最后一条 bullet)：□ 用得太多 → SM 上能同时驻留的 block 变少——共享内存是按 block 分摊的配额，占用率会掉 (Part I 的账)；□ 访问布局不好 → bank conflict，访问被串行化。

工厂话术照旧：这就是公共工作台——零件从大仓库领一次放到台上，全组围着用。

大白话：一张工作台，隔板可调：多少归你们班组的项目台，多少归自动整理的缓存架。

【问】可能被问：“L1 和共享内存的比例谁定？”→ 可以按 kernel 配置 (carveout 属性, cudaFuncSetAttribute 一行调用) ——知道有这个旋钮即可，不展开。

【问】可能被问：“bank 冲突到底是什么？”→ 共享内存物理上分成 32 个 bank (正好对应 warp 的 32 条通道)，同一拍里多个线程访问同一个 bank 的不同地址就得排队。全 warp 访问连续地址 (各落各的 bank) 或同一地址 (广播) 都没事。矩阵转置是经典踩坑场景，第 10 章有解法 (padding)。

第 33 页 | TMEM and TMA: Feeding the Tensor Cores * (20 秒跳过页)

【口播】这页标了星——Blackwell 的新组件，第 10 章整章细讲，今天 20 秒飞过，只要记两个名字大概是干啥的：TMEM——每 SM 一块 Tensor Core 专属的 SRAM，给矩阵乘当累加器，程序员的指针摸不到它；TMA——自动搬运工，你给它一张“从哪搬、搬多大”的描述符，它就在后台把数据块搬进搬出。合起来的效果：矩阵乘的中间结果从头到尾不用碰全局内存。第 10 章见。(跳下一页)

大白话：TMA 是在后台运货的叉车班组；大厨 (Tensor Core) 从头到尾不用离开灶台。

【问】若有人追问细节 → “第 10 章有完整的一章，包括 warp specialization 和 descriptor 的写法；今天先记住这两个角色。”

第 34 页 | Constant Cache: One-Cycle Broadcast (常量缓存: 单拍广播)

【口播】这页三句话讲完: **Constant Memory 是一小块全局只读内存** (64 KB 的 `__constant__` 空间), **每个 SM 前面配了一块约 8 KB 的 Constant Cache**。

它的**特殊能力**: 如果一个 warp 的 32 个线程**同时读取同一个地址**, 只需读取一次, 就能把值**广播**给全部 32 个线程——和读寄存器一样快。

但反过来, 如果 32 个线程**读取不同地址**, 请求会**分批甚至串行**处理。

所以结论看下面两个框: **适合**——小型、只读、**所有线程统一访问**的参数 (LLM 里的现成例子: RoPE 查找表、ALiBi 斜率、LayerNorm 的 γ/β 、量化 scale); **不适合**——普通的大 tensor, 或每个线程查**不同位置**的表。

大白话: **它是喇叭不是信箱: 一次广播全班 32 人都听到——前提是大家问的是同一个问题。**

【问】可能被问: “这些数据放共享内存不行吗?” → 行, 但亏: 共享内存要自己写代码搬、占 block 的工作台配额; 常量缓存的广播是硬件免费送的, 而且跨 block 全局有效。判断标准一条: 只读 + 全体同址 → 常量内存。

第 35 页 | L2 Cache: The GPU-Wide Middleman (L2: 全 GPU 的中间人)

【口播】往下走一层, 出 SM 了。L2 是**全 GPU 唯一一块所有 SM 共享的缓存: 126 MB** (这个数值得停一秒——比很多 CPU 的 L3 还大), 约 200 拍, 聚合带宽数 TB/s, 同时兜住 L1 放不下的溢出。

它最大的价值是**跨 block 复用**: block A 从 HBM 取过的数据, block B 再要时直接从 L2 拿——**不必再去 DRAM 走一趟**。想想 Part I 的实例: 3,907 个 block 处理同一个数组, 相邻 block 的数据边界高度重叠, L2 就是它们之间无声的共享层。

右框是书的提示, 也是第 7 章的预告: 把全局内存访问组织成 **128 字节对齐、可合并 (coalesced)** 的段, 与缓存行整齐对应——避免拆分事务, 同时榨满 L2 和 DRAM 带宽。

大白话: **L2 是全厂的中转仓: 别的小组已经从大仓库运来的零件, 你直接到中转仓取——前提是大家按“整托盘”下单, 别一颗螺丝一颗螺丝地要。**

【问】可能被问: “L2 能编程控制吗?” → 可以部分控制: CUDA 提供 access policy window (`cudaStreamSetAttribute` 设置 persisting 区域), 把某段地址“钉”在 L2 里——推理场景钉 KV cache 是典型用法。进阶话题, 知道存在即可。

第 36 页 | Global HBM3e, the Dual-Die B200, and Coherency (HBM、双芯粒与一致性)

【口播】阶梯的最底层，三个知识点：

□ **数字**：B200 上 **180 GB** (B300 约 288 GB)，带宽 **约 8 TB/s**——但延迟几百到 1000+ 拍，**整条链路上最慢的一环**。第 31 页说的寄存器溢出、还有超大的自动数组，付的都是这个价。

□ **冷知识 (书中注)**：B200 物理上是**两块光刻极限大小的裸片**，用 **10 TB/s** 的片间互连拼起来，各带 4 个 HBM3e 栈——但呈现给你的是一块 **GPU、一个地址空间**。10 TB/s 比 HBM 带宽还高，所以你基本感觉不到缝。

□ **一致性点 (point of coherency, 图 6-15)**：内存一致性按作用域逐级建立——**线程 → block → 簇 → 设备 → 系统**，通信范围越广、代价越高。这解释了 Part I 的设计哲学：为什么协作被限制在 block 内（工作台上一致性最便宜）、为什么跨 block 默认互不通气。

图 6-14 是全局内存的层级位置图。

大白话：**大仓库又大又能吞吐（一次能发很多货），但下一单等回货的时间很长——所以要么少下单（片上复用），要么让别的小队在等货时干活（占用率）。**

【问】可能被问：“双芯粒对我写代码有影响吗？”→书的态度：当一块 GPU 用，透明。严格说跨芯粒访问对方的 HBM 栈有轻微 NUMA 效应，但 10 TB/s 互连让它基本不可见；除非做极限调优，不用管。

第 37 页 | Unified Memory Hides Copies, Not Their Cost (统一内存：藏起来的是拷贝，不是代价) □

【口播】标题就是这页的全部：**统一内存隐藏的是拷贝这个动作，不是拷贝的代价。**

它是什么：cudaMallocManaged() 给你 **CPU+GPU 一个连贯的地址空间**——不用 h_/d_ 两份指针，不用手写 cudaMemcpy，Part II 六步流程里的两步拷贝直接消失，代码干净得多。

底层机制（图 6-16）：内存页在 CPU 和 GPU 之间**按需迁移**——谁访问、页搬到谁那边。

代价在哪：GPU 摸到一个还在 CPU 侧的页 → **缺页中断，kernel 当场停摆**，等页搬完才继续。走 PCIe 时，这种按需搬页**可能比手动 memcpy 还慢**（一页一页搬 vs 一大块连续搬）。

硬件缓解：Grace 超级芯片上的 **NVLink-C2C (约 900 GB/s)** 让迁移接近本地速度——**但延迟永远不为零**，“要用时才发现不在”这个结构性问题不变。

大白话：**统一内存是自动送料服务——省事，但小组要用时零件还没到车间，全组就只能站着等。**

【问】可能被问：“那生产上到底用不用？”→分场景：原型/科研期非常好用（代码量减半）；性能关键路径上要么回到手动管理，要么用下一页的 prefetch + advise 把代价治住。Grace 这类 C2C 平台上实用性大增。

第 38 页 | Taming Unified Memory: Prefetch, Advise, Attach (驯服统一内存)

【口播】上一页的病，这一页三味药，对应左边三段代码：

药一：预取。cudaMemPrefetchAsync(ptr, size, gpuId, stream)——在 kernel 需要之前把数据搬过去，把“首次触碰才缺页”变成**可与计算重叠的异步传输**（图 6-17）。这是最重要的一味，等价于“提前下单”。

药二：建议。cudaMemAdvise 三个提示：SetPreferredLocation(这块数据的首选住址)；SetReadMostly(只读为主 → 允许各处**复制只读副本**，谁读谁有，互不打架)；SetAccessedBy(让另一块 GPU **建立映射直接访问而不迁移**——数据不搬家，远程读，图 6-18)。

药三：绑定。cudaStreamAttachMemAsync(stream, ptr, 0, cudaMemAttachSingle)——把这段内存的缺页/迁移**圈在一条流里**，不让**其他流**被它拖着停。

补充：没有 NVLink-C2C 的平台，把数据钉在 **NUMA 本地**（对等拷贝/预取）。

底线（右栏最后一条）：三味药用齐，性能 ** 接近手动 cudaMemcpy**，同时保留统一内存的简洁——鱼和熊掌基本兼得。

大白话：**自动送料没问题，但要提前下单（prefetch）、写清用途（advise）、别让别的班组陪着等（attach）。**

【问】可能被问：“prefetch 和手动 memcpy 还有什么区别？”→ 功能上几乎等价，区别在心智模型：prefetch 是“优化提示”，忘了写程序照样对（只是慢）；memcpy 是“正确性义务”，忘了写就是 bug。统一内存 + prefetch = 默认正确、可选变快。

第 39 页 | Occupancy Ground Rules (占用率基本法则) □

【必说】**转场 Part III→IV（对应页面顶部灰字）**：“内存层级告诉我们 warp 为什么会 stall。现在回到 occupancy：一个 warp 在等数据时，我们需要多少其他 warp 才能填满空档？”

【口播】蓝框是书里“CUDA 性能最基本的法则”：“**Launch enough parallel work to fully occupy the GPU.**”——启动足够多的并行工作，把 GPU 填满。

下面两条规则务必分清，它们是这一部分的纲：

规则一——occupancy 低且性能差：第一味药是**加并行度**（更多 block/线程），加到“有足够多的 ready warp 能把延迟藏住”为止——**性能不再上涨（plateau）就停**，不要以某个固定百分比为目标（第 45 页 38.7% 拿到 22 倍就是证据）。

规则二——occupancy 已经中高但还是慢：如果 kernel 是 **memory-bound**，硬推到 100% **没用**——你只需要“刚好够藏延迟”的 warp 数，超过之后瓶颈在带宽，不在人手。

预告：接下来 6 页是一个完整案例——同一个操作（ $C = A + B$ ，一百万元素）、两种实现、两个 profiler 的裁决。

大白话：**规则一是让车间站满小组；规则二提醒你：车间站满了，卡在仓库运货卡车上照样停工。**

【问】可能被问：“occupancy 到底多少算够？”→ 没有普适数字——“够”的定义是**把停顿藏住、性能进入平台期**。案例里 38.7% 就够了；算术密集的 kernel 可能 25% 就饱和；实测为准。

第 40 页 | Case Study, Take 1: addSequential (案例上：串行版) □

【口播】案例第一回合，先看一个“反面教材”kernel。左边代码：addSequential——一个线程做全部一百万次加法。注意两处：函数体里 `if (blockIdx.x == 0 && threadIdx.x == 0)` 把活儿全锁给 0 号线程，一个 for 循环从头加到尾；启动配置是 `**<<<1, 1>>>**`——1 个 block、1 个线程。右边三个要点：□ 一个线程 = 一个 warp = 一个 SM 在干活——其余全体围观：B200 上一百多个 SM，一百多减一个全在闲置；□ 书里的评语（意译）：GPU 庞大的资源基本闲置，占用率极差、性能自然极低；□ 最致命的机制性问题：**没有其他 warp 可切换**——每次访存等待都变成纯粹的空转，Part I 讲的延迟隐藏完全失效，因为调度员手里只有一支队伍。

大白话：**你租下了整座工厂，然后只雇了一名工人。**

【问】可能被问：“谁会真写 `<<<1, 1>>>?`”→ 显式没人写，但**隐式的串行段很常见**：没并行化的预处理/后处理、按样本循环调 kernel、自定义算子里“先算个全局统计量”的串行小尾巴。这个极端例子是用来给 profiler 指标定标尺的。

第 41 页 | The Same Trap in PyTorch (同一个坑的 PyTorch 版) □

【口播】别以为用 PyTorch 就免疫了——同一个坑换了件衣服。左边代码：Python 里 `for i in range(N): C[i] = A[i] + B[i]`，对一百万个元素逐个下标赋值。

三个要点：□ 这不是“一个 kernel 跑一百万次循环”，而是**串行发射一百万个迷你 kernel**——每次 `C[i] = A[i] + B[i]` 都是一次独立的 GPU 操作，书里的说法：GPU 沦为“一个标量的、毫无并行的处理器”；□ 占用率和上一页一样惨，还要**外加每次 launch 的 CPU 开销 × 1,000,000**——CPU 提交开销通常几微秒，光发射就要几秒；□ 书的建议：几乎总能找到 **PyTorch 原生的向量化写法**（包括 `torch.compile` 自动生成的）——下一页就是。

这一页对我们组是最实用的一页：**Python 循环套 GPU 逐元素操作，是生产代码里最常见的性能事故。**

大白话：**最常见的 GPU 性能 bug：一不小心把 GPU 当成了一颗非常昂贵的单核 CPU。**

【问】可能被问：“怎么发现自己踩了这个坑？”→ `nsys` 时间线一眼可见：**密密麻麻的微型 kernel**，每个之间还有 CPU 提交间隙，GPU 利用率个位数。修法：改写成张量化操作，或交给 `torch.compile`。

第 42 页 | Case Study, Take 2: addParallel —and C = A + B (案例下：并行版) □

【口播】正确写法两种形态。左边 CUDA 版 `addParallel`：**一个线程管一个元素**——就是 Part II 学的标准骨架（算 `idx`、边界检查、干活），启动 `<<<(N+255)/256, 256>>>`。注意参数上的新面孔 `**__restrict__`：程序员向编译器承诺“这几个指针互不重叠（无别名）”，编译器就敢放心地缓存和重排访存——解开优化的手脚。

左下小字别漏：书里的完整版本还用了**锁页主机内存、非阻塞流、`cudaMallocAsync/cudaMemcpyAsync`**——正是 Part II 养成的全部习惯，这里全用上了。完整版**下一页顺手带过**——这页先聚焦“一个线程 vs 一百万个线程”这个对比本身。

右边三个要点：□ 约 3,907 个 block × 256 线程——**一百万个线程**铺满数组（图 6-19），Part I 实例页的配置原样重现；□ PyTorch 等价写法就一行：`**C = A + B**`——单个向量化 kernel，同时调动海量线程；□ 对比上一页：同样的数学，写法一变，GPU 从“单核 CPU”变回吞吐量机器。

大白话：**还是那座工厂，现在每个工位都有人——PyTorch 版等于直接“找劳务派遣公司包了”。**

【问】可能被问：“`__restrict__` 到底干了什么？”→ C99 的 `restrict` 在 CUDA 里的写法。没有它，编译器必须假设 A、B、C 可能指向同一块内存（写 C 可能改掉 A），于是每次都老老实实重新加载；有了它，编译器可以把数据留在寄存器、合并访存、重排指令。逐元素 kernel 的惯用签名。

第 43 页 | addParallel, Full Version: The Part II Habits Assembled (完整版: Part II 习惯大集合)

【口播】顺手带过，30-40 秒，不逐行念。上一页的 kernel 一个字没变，这页只是给主机侧换上正式工装——**每一行都是 Part II 教过的**：建非阻塞流；cudaMallocHost 锁页分配（六步流程第 1 步）；cudaMallocAsync 从池里取显存（第 28 页）；然后是唯一的新面孔 ****cudaMemcpyAsync——异步拷贝**，排进流 s 里 **，它能与计算重叠的前提正是主机缓冲区是锁页的（第 23 页的伏笔）；kernel 也发进同一条流；cudaStreamSynchronize(s) **只等这一条流**，不做全局同步；最后 cudaFreeAsync 异步归还。

一句话：同一个 kernel，生产级的管线——Part II 的习惯在这里全部组装完毕。

大白话：同一个 kernel，换上正式工装——Part II 的习惯全套上身。

第 44 页 | Measuring It: nsys and ncu (怎么量：两件裁判工具)

【口播】空口无凭，两件工具当裁判，各管一摊：

nsys (Nsight Systems) ——整机秒表。命令：nsys profile --stats=true -t cuda,nvtx -o report < 程序 >。它回答“**时间花到哪儿去了**”：GPU 是没喂饱 (starved) 还是被堵住 (blocked)? kernel 之间有没有空档?

ncu (Nsight Compute) ——单 kernel 显微镜。命令里那个超长的指标 sm__warps_active.avg.pct_of_peak_just_achieved_occupancy **本尊**——实测占用率在工具里的学名。它回答“**这个 kernel 为什么慢**”：占用率? 停顿? 缓存未命中?

为什么两个都要跑：只跑 nsys 看不到 kernel **内部**的低效；只跑 ncu 看不出 kernel 之间数据喂得够不够快。先 nsys 找到时间大头，再 ncu 解剖那个大头——标准工作流。

大白话：**nsys 是给整台机器计时的秒表；ncu 是对准单个 kernel 的显微镜。**

【问】可能被问：“profiler 本身拖不拖慢程序?” → nsys 开销小 (时间线采样)；ncu 开销大——它会**重放 (replay)** kernel 多次来采不同计数器，所以用 --kernel-name 只对目标 kernel 下手，别全量跑。

第 45 页 | Parallelism Cuts Runtime 22× at Only 38.7% Occupancy (裁决: 22 倍) □

【口播】裁决书（表 6-6，示意值，真实数据在书的 GitHub 仓库）。四行逐行读：

Kernel 耗时：48.21 ms → **2.17 ms**, **22 倍**——本章标题“Maximizing Occupancy”的兑现。

GPU 利用率：1.5% → 95%。

实测占用率：1.3% → **38.7%**。

Warp 执行效率：3.1% → 100%——3.1% 这个数很有讲头： $3.1\% \approx 1/32$ ，串行版每个 warp 只有一条通道在干活，正是“一个人的赛艇队”的数字化呈现。

右图（图 6-20）是加速的本质：串行版时间线是“操作—空档—操作—空档”；并行版**空档被其他 warp 的工作填满**——Part I 延迟隐藏的可视化终章。

两个洞察：□ **38.7% 就买到了 22 倍——根本不需要 100%**（回收第 2 页规则一的“不设固定目标”）；□ 表里四个指标——utilization、SM active、occupancy、warp efficiency——**相关但不同维度**：95%、38.7%、100% 能同时出现，正说明它们量的不是一件事。

大白话：**38.7% 的上座率就赚回 22 倍——先把人招够，别急着追满座。**

【问】可能被问：“为什么 38.7% 就够了？”→ 回到 occupancy 的定义（第 17 页）：它是“调度员手里有多少候选”。 $38.7\% \times 64 \approx 25$ 个常驻 warp/SM，对这个访存模式已经足够在每次停顿找到就绪者——停顿被藏住后，再加 warp 就没增量了。

第 46 页 | A Busy GPU Can Still Be Waiting on Memory (LLM Decode) (忙碌的 GPU 仍可能在等内存) □

【口播】顶部灰字是全场最重要的转场：“并行版达到 95% utilization、只有 38.7% occupancy、已经快了 22 倍——所以我们不需要 100%。但它到 GPU 算力峰值了吗？**仍然没有，因为 vector add 主要在搬数据。**”

三个要点：□ 并行度拉满之后，下一个抓手是单 warp 效率（ILP 等，第 8 章）——但**哪怕 occupancy 100%**，只要 kernel 是 **memory-bound**（受制于数据搬运而非计算），性能照样上不去；□ 书里的经典例子：**LLM 的 decode 阶段**——每生成一个 token，都要把**模型权重**从 HBM 整个流进片上；几千亿参数 $\times$ 约 1 字节 $\approx$ **每个 token 几百 GB** 的搬运量——多少线程都救不了带宽；□ 蓝框是让情况雪上加霜的趋势：**GPU 算力增速超过内存带宽**——HBM3e 约 8 TB/s，但算力和模型规模涨得更快，所以“优化数据搬运”在现代 AI 负载里绝对关键。

这页把听众从“occupancy 万能”的错觉里拽出来，为 Part V 的 roofline 铺路。

【必说】**收尾交代：**“书里这一节还有两件占用率调优工具——__launch_bounds__ 和 Occupancy API（p257-259），时间关系今天不讲，材料在共享仓库，感兴趣会后自取。”

大白话：**当整个活儿就是“从仓库搬箱子”，前台多雇几个职员没有用——这就是 Part V 屋顶线要形式化的事。**

【问】可能被问：“为什么 decode 每个 token 都要读全部权重？”→ 自回归生成一次只算一个 token，矩阵乘退化成“向量 $\times$ 矩阵”，每个权重只被用一次（复用率 = 1）——读进来做一次乘加就扔。batch 加大才能摊薄（多个序列共享同一遍权重读取），这正是第 15-18 章 continuous batching 的动机。

第 47 页 | Roofline Tells Us Which Resource Is the Limit (屋顶线模型) □ 建议讲 2-3 分钟

【必说】转场 Part IV→V (对应页面顶部灰字): “不能再盲调 block size 了——最后一个问题: 撞上的是 compute ceiling, 还是 memory ceiling?” 顺带一句交代: “书里这里还有一小节 Compute Sanitizer (正确性排查四件套, p259), 时间关系跳过——记住工具名, 材料在共享仓库。”

【口播】全章的“指南针”到了。三步讲清:

第一步, 定义横轴: 算术强度 (arithmetic intensity) = 每从 HBM 搬 1 字节, 做多少次浮点运算 (FLOPs/Byte) ——一个 kernel 的“体质”, 和硬件无关。

第二步, 画两道屋顶: 平的算力屋顶 (FP32 约 80 TFLOP/s——你算得再密也超不过算力上限) 和斜的内存屋顶 (约 8 TB/s——搬得再多也超不过带宽上限)。两道屋顶交汇处叫**屋脊点 (ridge point): $80T \div 8T \approx 10 \text{ FLOPs/Byte}$** 。落点在屋脊左边 = memory-bound (内存受限), 右边 = compute-bound (算力受限)。

第三步, 现场算我们的案例: $C = A + B$ ——读 A、B 共 8 字节, 加 1 次, 写 C 4 字节 $\rightarrow 1 \text{ FLOP} \div 12 \text{ Byte} \approx 0.083$ ——离屋脊点差 100 多倍, 无可救药地 memory-bound。这就从数学上解释了第 46 页: 为什么占用率救不了它——图 6-21 上它就钉在最左边的斜坡上。

书中注: LLM **两头都占——prefill 偏 compute-bound** (长序列一次算完, 权重充分复用), **decode 偏 memory-bound** (一次一个 token, 权重复用率 1) ——第 15-18 章的主线。

收尾预告: “横轴听着抽象? 下一页用七个例子把它讲实。”

大白话: **屋顶线先问一句: 这个 kernel 是饿数学, 还是饿数据? 答错方向, 白干。**

【问】可能被问: “80 TFLOP/s 哪来的? Tensor Core 不是几 P 吗?” $\rightarrow$ 80T 是 FP32 CUDA core 的示意峰值。换用 Tensor Core + 低精度, **算力屋顶整体抬高、屋脊点右移——所以“换精度”等于“换屋顶”, 正好是下一页。**

第 48 页 | Arithmetic Intensity by Example: Reuse Moves You Right (算术强度实例表) □ 建议讲 2 分钟

【口播】先把横轴的定义再钉一遍：**AI = 整个 kernel 的总 FLOPs ÷ 从 HBM 读写的总字节数**——是总量统计，不是“某个字节被算了几次”。数 FLOP 的规矩：加法 1、乘法 1、**FMA ($a \times b + c$) 算 2**。然后带大家读表，从上到下：

前三行全是 memory-bound：向量加法 $1 \text{ FLOP} \div 12 \text{ B} = \mathbf{0.083}$ ；原地乘 2 是 $1 \div 8 = 0.125$ ；SAXPY ($y = a \cdot x + y$) 是 $\text{FMA } 2 \text{ FLOPs} \div 12 \text{ B} = 0.167$ ——标量 a 在寄存器/常量缓存里，HBM 流量可忽略。规律：**逐元素操作怎么写都在 0.1 左右**，离屋脊点 10 差两个数量级。

第四行开始翻盘——靠复用：同一个 x 在寄存器里迭代 100 轮 ($x = x \cdot a + b$)，HBM 只进出 8 B，却做了 200 FLOPs $\rightarrow \mathbf{AI = 25}$ ，过了屋脊点。关键： **$A[i]$ 只从 HBM 读一次**，之后 100 轮全在寄存器里。

第五行矩阵乘： $N \times N$ 矩阵乘 $\approx 2N^3 \text{ FLOPs}$ (每次乘加 2)；tiling 做得理想时 HBM 流量 ≈ 3 个矩阵各读写一遍 $= 12N^2 \text{ B} \rightarrow \mathbf{AI \approx N/6}$ 。 $N=4096$ 就是 **683**——这就是矩阵乘天生 compute-bound 的数学原因： **A 的一行要和 B 的很多列相乘**，一份数据天然要被用 N 次；tiling (Part III 第 32 页的 load once, reuse many) 就是把这个复用真正兑现的手段。

最后两行是我们天天打交道的：decode——每个 FP16 权重 2 B 只做一次乘加 2 FLOPs $\rightarrow \mathbf{AI \approx 1}$ ，memory-bound (这就是第 46 页的定量版)；**prefill**——128 个 token 共享同一次权重读取 $\rightarrow 256 \text{ FLOPs} \div 2 \text{ B} \approx 128$ ，compute-bound。**batch 就是 LLM 世界的复用**。

下面三个 bullet 收口：□ 0.083 的带宽上限 $= 8 \text{ TB/s} \times 0.083 \approx \mathbf{0.67 \text{ TFLOP/s}}$ ，只有算力屋顶的 1%——occupancy 救不了向量加法的定量证明；□ 向右走的唯一引擎是**复用**：寄存器 (第 4 行)、tiling (第 5 行)、batch (第 7 行)；□ 分母**只数 HBM 流量**，L2/共享内存内部的命中不算——同一算法写得烂 (每次乘加都回 HBM) AI 会远低于 $N/6$ ，所以要信 profiler 的**实测 AI**，不是纸面推导。

大白话：**同一份数据、更多次计算——复用 (寄存器、tiling、batch) 是把你推向右边的唯一引擎**。

【问】可能被问：“FMA 为什么算 2 FLOPs？” $\rightarrow$ 一条指令干了乘和加两件事，业界惯例按 2 计——GPU 的峰值 TFLOPS 也是按 $\text{FMA} \times 2$ 标的。

【问】“L2 里反复读的数据算不算？” $\rightarrow$ 不算——这张 roofline 的分母是 HBM 一级；也可以对 L2/L1 各画一张 roofline，分母换成对应层的流量。

【问】“decode 那行怎么没算 KV cache？” $\rightarrow$ 粗略模型只看权重流量；算上 KV 读取分母更大、AI 更低——结论 (memory-bound) 只会更强。

第 49 页 | Moving Right on the Roofline: Lower Precision Pays Twice (低精度一鱼两吃)

【口播】知道自己 memory-bound 之后怎么办？往屋脊点的**右边**挪——让每个字节干更多活。三条路：数据留在片上复用（Part III 的本事）、算子融合（少走几趟 HBM）、或者最直接的——**把字节变小**。

算一笔账：一次 128 字节的内存事务，能装 **32 个 FP32 = 64 个 FP16 = 128 个 FP8 = 256 个 FP4**。FP32 → FP16，同样的带宽下数据量翻倍，**FLOPs/Byte 立刻翻倍**；FP8 相对 FP16 再翻一倍。这就是“一鱼两吃”：**低精度既是算力优化（Tensor Core 低精度吞吐更高），也是带宽优化（同样字节装更多数）**——而对 memory-bound 的负载，第二吃才是主菜。

Blackwell 的硬件加持：原生 **FP8 和 FP4 Tensor Core**，外加**硬件解压**——权重以压缩形态存在 HBM 里（甚至比 FP4 更狠的 4 比特/2 比特方案），读取时硬件即时展开、再转成 FP16/FP32 做精确累加。净效果：**等效内存带宽变大**——对 memory-bound 的 token 生成是体系结构级的礼物。

蓝框要点一句话：**字节减半 → 算术强度翻倍 → 向算力屋顶靠近**。

大白话：**装货的卡车没变，把每件货做小一半，一趟就能拉两倍的活儿**。

【问】可能被问：“压缩权重会损精度吗？”→ 分两层：**量化本身**（FP16→FP8/FP4）有损，靠训练侧/校准控制；**硬件解压那一步**是对已量化数据的无损展开，且累加用 FP16/FP32 做——不额外叠加误差。

第 50 页 | Profiling Workflow: Diagnose, Fix, Re-measure (诊断—修复—复测)

【口播】全章方法论收口：怎么用 profiler 把前面所有知识串成一个循环。

诊断特征：memory-bound 在 ncu 里的指纹是 **DRAM 利用率高 + ALU 利用率低**——warp 都在等内存；同时留意 global load efficiency（全局加载效率）下滑——那是访存不合并的信号。nsys 时间线上 kernel 之间的**空档** = GPU 在等数据传输。

两个惯犯：期望“拷贝与计算重叠”却看到串行排队？□ 不小心用了**默认流**（自带隐式全局同步——Part II 第 7 页为什么建非阻塞流的回收）；□ **忘了用锁页内存**——没有 pinned, cudaMemcpyAsync 名字里有 Async 也**无法真正重叠**（第 23 页的伏笔在这兑现）。

修复后的样子：右图（图 6-22）上下对比——同步版“拷贝、算、拷贝”排队走；异步版三者叠着走，空档消失，访存管道利用率逼近峰值。

右下斜体是全章最后一条纪律：**每改一处就复测一次**——profiler 确认停顿真的减少了再动下一处。工具的新特性顺嘴提：ncu 的 range replay、源码关联——知道有就行。

大白话：**修车不能凭感觉换零件——每换一个，上路测一圈**。

【问】可能被问：“global load efficiency 低说明什么？”→ 请求的字节数 ÷ 实际搬运的字节数低——访存不合并，warp 的 32 个请求散落在很多缓存行里，每行只用了几个字节。解法是第 7 章的 coalescing。

第 51 页 | Key Takeaways (书中八条要点)

【口播】书的八条要点，我们按四对念，每对一句话（这页 90 秒，别逐条展开——都讲过了）：

执行模型一对：SIMT——32 线程的 warp 齐步走，大量 warp 在飞 = 延迟被藏 (Part I)；层级——线程 → block ($\leq 1,024$) → grid, `__syncthreads()` 能少用就少用 (Part I/II)。

资源一对：占用率与上限——取 32 的倍数；每 SM: 64 warp、32 block、228 KB 共享内存、每线程 255 寄存器 (Part I 上限表)；启动参数——256 起步、网格向上取整、profiling 定终值 (Part II/IV)。

内存一对：异步分配——MallocAsync + 流 + 内存池，PyTorch 缓存分配器同理 (Part II)；内存阶梯——上层复用 (寄存器/共享/L1)，下层合并 (L2/HBM)；统一内存用 prefetch + advise 驯服 (Part III)。

方向一对：roofline——FLOPs/Byte 决定主战场；FP16/FP8/FP4 + 硬件解压向右挪；TMEM + UMMA 能把 kernel 从内存受限拉回算力受限 (Part V)。

大白话：八条要点 = 四对钥匙：怎么执行、给多少资源、数据放哪、往哪个方向调。

第 52 页 | Conclusion and What's Next (总结与下一步)

【口播】蓝框一句话总结全章：让 GPU 忙起来 (occupancy)、让数据靠得近 (memory hierarchy)、让 roofline 告诉你下一仗打哪。

书的收尾提醒（也是第 17 页 occupancy 定义页埋的伏笔，现在收回来）：最大占用率并不总是最优——单线程 ILP 足够时，中低占用率也能赢；有时“线程更少、每人寄存器更多”反而更快。一切以实测为准——这句话是全章的底色。

预告后面的书：第 7-8 章讲 warp 分化与访存调优（我们埋的好几个“第 7/8 章见”在那兑现）、第 9 章算术强度、第 10 章 TMA 与 warp 专业化。

最后是我的预告：**我的第二讲——第 12 章**——建立在本章之上：单个 kernel 快了之后，怎么编排它们，让 GPU 永远不用等 CPU？到时见。

(Questions? 停在这页答疑——结论页留在屏幕上，别切到谢谢页。)

大白话：全章三个动词：填满、靠近、看罗盘。

【问】可能被问：“ILP 足够时低占用为什么能赢？”→ 回忆 Part I“发射 vs 执行”的问答：同一 warp 的独立指令可以背靠背发射。每线程干的独立活儿多 (ILP 高)，少量 warp 也能把管线喂饱，还省下寄存器让每线程拿更多——Volkov 经典结论，书里点到为止。

第 53 页 | References & Further Reading (参考文献)

【口播】不逐条念。口头一句：“引用主要四类——原书第 6 章、NVIDIA 的架构调优指南和 CUDA 编程指南、roofline 原始论文 (Williams 等, CACM 2009)、以及各工具的官方文档；本 deck 示意表格的真实基准数据在书的 GitHub 仓库。感谢大家，回到上一页提问。”